[ПОЛНОЕ НАИМЕНОВАНИЕ УЧЕБНОГО ЗАВЕДЕНИЯ]

[Наименование факультета / института]

[Наименование кафедры]

ОТЧЁТ

по лабораторной работе № 3

«Потоковая обработка данных: Apache Kafka и Spark Structured Streaming»

по дисциплине «[Название дисциплины]»

Индивидуальный вариант № 2 — банковские транзакции

Выполнил(а): студент(ка) группы [Номер группы]

[Фамилия Имя Отчество]

Проверил(а): [должность, учёная степень]

[Фамилия И. О. преподавателя]

[Город] [Год]

СОДЕРЖАНИЕ

1. Цель работы.....3
2. Постановка задачи.....3
3. Описание варианта № 2.....4
4. Архитектура решения.....4
5. Конфигурация Aiven и Kafka Topic.....6
6. Пакетная обработка.....7
7. Статистический анализ (пакетный этап).....9
8. Потоковая обработка.....10
9. Сравнение первоначальных и обновлённых результатов.....12
10. Проверка доставки сообщений в Aiven.....13
11. Выводы.....15
12. Ответы на контрольные вопросы.....16
Список использованных источников.....24

1. Цель работы

Цель работы — освоить потоковую обработку данных с брокером сообщений Apache Kafka и Spark Structured Streaming:

- сгенерировать и пакетно обработать исторический набор банковских транзакций;
- организовать защищённую (SSL) передачу новых транзакций через управляемый сервис Kafka в Aiven;
- обработать их в Spark Structured Streaming микропакетами;
- объединить с историческими данными и показать автоматическое обновление аналитических показателей;
- сохранить результаты в облачном хранилище Google Drive.

2. Постановка задачи

Необходимо:

1. Сгенерировать воспроизводимый синтетический набор банковских транзакций (около 500 000 строк) с пропусками, дубликатами и некорректными значениями и сохранить его в Google Drive.
2. Выполнить пакетную обработку в PySpark: явная схема, проверка качества, очистка, описательная статистика, топ-5 категорий, группировки по городам, валютам и типам операций.
3. Создать сервис Apache Kafka в Aiven и топик, настроить SSL-подключение из Google Colab для Python-продюсера и для Spark.
4. Отправить порцию новых транзакций в топик и прочитать их в Spark Structured Streaming (`readStream`) с проверкой, обработкой некорректных сообщений и регулярным обновлением результатов.
5. Объединить потоковые транзакции с историческими и сравнить показатели до и после поступления новых данных.
6. Сохранить итоговые и промежуточные результаты и журнал в Google Drive, подтвердить доставку сообщений в Aiven UI.

Вычислительная среда — **Google Colab**: одна виртуальная машина, Spark в локальном режиме `local[*]`. Это не Google Cloud Dataproc и не распределённый кластер. Брокер — **Aiven for Apache Kafka**, хранилище — **Google Drive**.

3. Описание варианта № 2

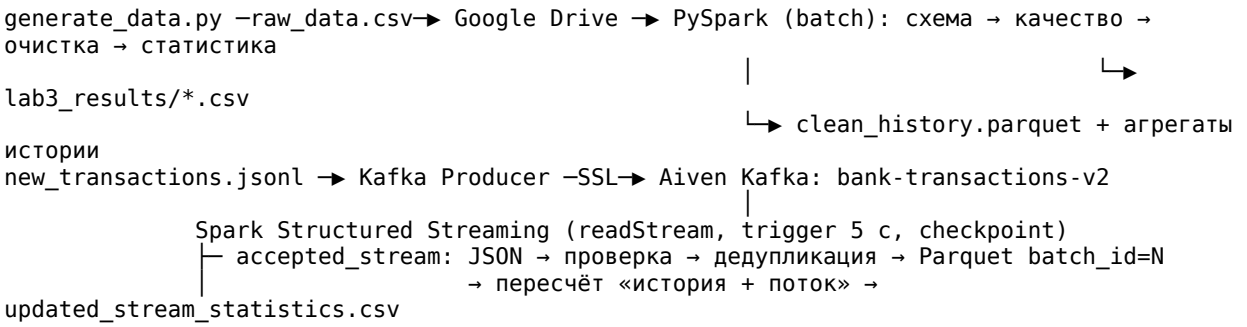
Тема — банковские транзакции. Обязательные поля: `transaction_id`, `client_id`, `transaction_type`, `amount`, `currency`, `city`, `device_type`, `timestamp`. Добавлено поле `fee` (комиссия банка) — второй осмысленный числовой признак.

Таблица 1 — Структура набора данных

Поле	Тип в CSV / JSON	Значения (синтетические)
`transaction_id`	строка	`T000000001…` (история), `S-<пакет>-000001…` (поток)
`client_id`	строка	`C0000001…`, ~20 000 клиентов
`transaction_type`	строка	payment 42 %, transfer 24 %, withdrawal 16 %, deposit 12 %, refund 6 %
`amount`	число	логнормальное распределение по типу операции и городу, в валюте операции
`currency`	строка	RUB 62 %, USD 15 %, EUR 14 %, CNY 9 %
`city`	строка	12 городов России (Moscow 30 %, Saint Petersburg 16 %, …)
`device_type`	строка	mobile, desktop, atm, pos_terminal, tablet
`timestamp`	строка `yyyy-MM-dd HH:mm:ss`	2025 год (история), с 01.01.2026 (поток); суточный профиль с пиком днём
`fee`	число	transfer 0,5 %, withdrawal 1 %, остальные 0

Для сопоставимости сумм в разных валютах при очистке вычисляются `amount_eur` и `fee_eur` по фиксированным **условным** курсам (RUB 0,0105; USD 0,92; EUR 1,00; CNY 0,13 EUR). Это не рыночные котировки.

4. Архитектура решения



```
            └─ stream_type_counts: groupBy(transaction_type), outputMode complete → memory
```

Компоненты и среда выполнения:

Таблица 2 — Компоненты решения

Компонент	**Технология**	**Где выполняется**
Генератор данных	Python (`generate_data.py`), seed 42	Colab (или Arch Linux)
Пакетная обработка	PySpark 4.0.4, `local[*]`	Colab
Брокер сообщений	Aiven for Apache Kafka, тариф Free	облако Aiven
Продюсер	confluent-kafka 2.16.0 (`producer.py`)	Colab (или Arch Linux)
Потоковая обработка	Spark Structured Streaming + `spark-sql-kafka-0-10_2.13:4.0.4`	Colab
Хранилище	Google Drive (`MyDrive/lab3_bank/`)	облако Google

Версии проверены на совместимость:

- Colab: Python 3.13, OpenJDK 21, предустановлен PySpark 4.0.4;
- Spark 4.0.4 работает с Java 17/21 и Python 3.9–3.13;
- версия коннектора Kafka обязана совпадать с версией Spark (4.0.4), суффикс — с версией Scala (2.13). Коннектор использует `kafka-clients 3.9.1`.

Модель объединения истории и потока. Простое объединение (`union`) статического и потокового DataFrame не сохраняет историю между перезапусками. Поэтому:

- очищенная история сохраняется в Parquet;
- принятые потоковые транзакции дописываются в устойчивое хранилище `stream_transactions/batch_id=N`;
- после каждого микропакета итоговые показатели пересчитываются как сумма агрегатов истории и агрегатов всех сохранённых потоковых транзакций.

Повтор микропакета перезаписывает только свой каталог, а повторы транзакций отбрасываются по `transaction_id`.

Хранение. Spark пишет checkpoint и хранилище потока на локальный диск ВМ Colab (`/content/lab3_work`). После остановки потоков каталог копируется в Google Drive (`lab3_intermediate/work_backup`), а в новой

сессии восстанавливается. Так checkpoint не зависит от особенностей файловой системы Google Drive и при этом переживает перезапуск среды.

5. Конфигурация Aiven и Kafka Topic

Сервис создан в Aiven Console:

1. Services → Create service → Aiven for Apache Kafka → Service tier: Free;
2. после перехода в состояние Running создан топик (Topics → Create topic).

Таблица 3 — Конфигурация сервиса и топика

Параметр	**Значение**
Сервис / тариф	[ЗАПОЛНИТЬ: имя сервиса], Free
Облако / регион	[ЗАПОЛНИТЬ: из Overview]
Версия Kafka	[ЗАПОЛНИТЬ: из Overview]
Способ аутентификации	клиентский сертификат (mTLS): `ca.pem`, `service.cert`, `service.key` [или SASL/SCRAM — указать]
Имя топика	`bank-transactions-v2`
Число партиций	2 [ЗАПОЛНИТЬ: проверить в UI]
Фактор репликации	[ЗАПОЛНИТЬ: значение из UI]
Retention period	[ЗАПОЛНИТЬ: значение из UI; на Free задаётся сервисом]
Максимальный размер сообщения	[ЗАПОЛНИТЬ: max.message.bytes из UI]
Пропускная способность тарифа	до 250 КиБ/с (ограничение Free)

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/01_aiven_service_running.png]

Рисунок 1 — Сервис Aiven for Apache Kafka в состоянии Running

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/02_topic_config.png]

Рисунок 2 — Настройки топика bank-transactions-v2

SSL-подключение.

- Python-продюсер (librdkafka) использует PEM-файлы напрямую: `security.protocol=SSL`, `ssl.ca.location`, `ssl.certificate.location`, `ssl.key.location`.
- Для JVM-процесса Spark из PEM создаются хранилища в формате PKCS12:
  - truststore с CA — `keytool -importcert -storetype PKCS12`;
  - keystore с сертификатом и ключом — `openssl pkcs12 -export`.

- Spark получает параметры `kafka.security.protocol=SSL`, `kafka.ssl.truststore.*`, `kafka.ssl.keystore.*` (тип PKCS12), `kafka.ssl.key.password` и `kafka.ssl.endpoint.identification.algorithm=https`.

Перед созданием хранилищ проверяется, что:

- файлы существуют и имеют формат PEM;
- срок действия сертификатов не истёк;
- закрытый ключ соответствует сертификату (совпадение открытых ключей);
- сертификат подписан CA.

Секреты загружаются из «Секретов» Colab или окном загрузки файлов в `/content/kafka_secrets` (права 700/600) и в Google Drive не сохраняются. Пароли хранилищ генерируются случайно на сессию и передаются утилитам через переменные окружения. Содержимое ключей и пароли не выводятся.

6. Пакетная обработка

Результаты в разделах 6–9 получены в **контрольном прогоне** всего блокнота на эквивалентном локальном стенде: Spark 4.0.4, Java 21, брокер Kafka 3.9.1 с SSL и обязательной клиентской аутентификацией. Генерация и очистка детерминированы (seed 42), поэтому при тех же параметрах пакетные результаты в Colab должны совпасть. [ЗАПОЛНИТЬ: после запуска в Colab сверьте числа с файлами `lab3_results/*.csv` и замените их при расхождении.]

Чтение и схема. CSV читается из Google Drive с заголовком и **явной схемой**: все 9 полей — `StringType`. Это намеренно: в исходных данных есть нечисловые суммы и испорченные даты, а в Spark 4 по умолчанию включён режим ANSI, при котором обычный `CAST` прервал бы задание. Типы приводятся безопасно функциями `try_cast(... AS DECIMAL(18,2))` и `try_to_timestamp` (форматы `yyyy-MM-dd HH:mm:ss` и ISO с `T`). Итоговая схема: суммы `DECIMAL(18,2)`, время `TIMESTAMP` (UTC). Исходный файл содержит **505**

000 строк и **9 столбцов** (500 000 базовых строк и 5 000 добавленных дубликатов).

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/03_batch_processing.png]

Рисунок 3 — Запуск пакетной обработки: схема, первые строки, число строк

Таблица 4 — Отчёт о качестве исходных данных (`data_quality_report.csv`)

Проверка	**Строк**	**Доля, %**
Пропуски `transaction_id` / `client_id` / `transaction_type` / `amount`	1 259 / 1 261 / 1 242 / 1 235	≈ 0,25 каждого
Пропуски `currency` / `city` / `device_type` / `timestamp`	1 307 / 1 289 / 1 198 / 1 299	≈ 0,25 каждого
Полные дубликаты строк	3 558	0,70
`transaction_id`, встречающиеся более одного раза	4 423	—
Сумма: не число / ≤ 0 / больше 10 000 000	513 / 1 066 / 497	0,10 / 0,21 / 0,10
Время: не разбирается / вне 2025 года / формат ISO (исправимо)	488 / 502 / 3 403	0,10 / 0,10 / 0,67
Валюта: неизвестная / неверный регистр (исправимо)	521 / 3 270	0,10 / 0,65
Город: неизвестный / требует нормализации	536 / 4 817	0,11 / 0,95
Тип операции / устройство: неизвестные значения	494 / 508	0,10 / 0,10

Правила очистки.

1. Удаляются полные дубликаты.
2. Строка отклоняется, если нет `transaction_id`, `client_id`, суммы, времени или валюты, а также если сумма не число, не больше 0 или превышает 10 млн, время не разбирается или вне периода, валюта неизвестна.
3. Каждая строка получает **одну** причину (первую сработавшую), поэтому двойного учёта нет.
4. Среди оставшихся при повторе `transaction_id` остаётся одна запись (детерминированно).
5. Не удаляются, а исправляются:
   - регистр и пробелы;
   - формат даты;
   - неизвестный или пропущенный город → `Unknown`;
   - тип или устройство → `unknown`.

Таблица 5 — Результат очистки (`cleaning_summary.csv`)

Шаг	**Строк**
Исходных строк	505 000
Удалено полных дубликатов	3 558
Отклонено: нет валюты / времени / клиента / id / суммы	1 294 / 1 285 / 1 256 / 1 251 / 1 230
Отклонено: сумма ≤ 0 / неизвестная валюта / сумма не число	1 060 / 515 / 506
Отклонено: время вне периода / сумма > 10 млн / время не разбирается	494 / 491 / 478
Удалено повторов `transaction_id`	1 408
После очистки	**490 174**

Баланс строк сходится: сумма удалённых и оставшихся строк равна 505 000. Исправлено: город → `Unknown` у 1 789 строк, нормализован город у 3 934, регистр валюты — у 3 195, формат даты — у 3 312.

7. Статистический анализ (пакетный этап)

Таблица 6 — Числовая статистика (`numeric_statistics.csv`), 490 174 строки

Столбец	**Среднее**	**Медиана**	**Ст. отклонение**	**Мин.**	**Макс.**
`amount` (в валюте операции)	6 885,41	1 359,33	19 624,70	0,50	1 459 778,70
`fee` (в валюте операции)	20,33	0,00	64,31	0,00	5 025,94
`amount_eur`	113,92	40,53	251,55	0,17	15 699,61

Из числовой статистики исключены:

- `transaction_id` и `client_id` — идентификаторы, среднее для них бессмысленно;
- `timestamp` — для времени приводятся минимум и максимум.

`amount` смешивает валюты, поэтому сопоставимой величиной служит `amount_eur`. Распределение сумм сильно скошено вправо: среднее почти втрое выше медианы.

Таблица 7 — Топ-5 значений категориальных столбцов (`top_categories.csv`)

Столбец	**Топ-5 (доля, %)**
`transaction_type`	payment 41,83; transfer 24,03; withdrawal 15,92; deposit 11,91; refund 5,95
`currency`	RUB 62,43; USD 15,05; EUR 13,69; CNY 8,83 (всего 4 значения)
`city`	Moscow 29,40; Saint Petersburg 15,94; Novosibirsk 8,09; Yekaterinburg 7,84; Kazan 6,99
`device_type`	mobile 46,04; atm 21,93; desktop 18,44; pos_terminal 8,65; tablet 4,60

Таблица 8 — Группировка по городам (`city_statistics.csv`)

Город	Транзакций	Сумма, EUR	Средняя, EUR
Moscow	144 087	19 084 829,27	132,45
Saint Petersburg	78 133	9 268 770,06	118,63
Novosibirsk	39 632	4 019 600,97	101,42
Yekaterinburg	38 446	4 027 168,41	104,75
Kazan	34 268	3 519 214,94	102,70
… (всего 12 городов и Unknown)			

По типам операций наибольший средний чек у deposit (385,43 EUR) и transfer (172,28 EUR), наименьший — у refund (20,89) и payment (28,92). По валютам рассчитаны количество, сумма и средняя в исходной валюте (`currency_statistics.csv`). Комбинированная группировка «город × валюта × тип операции» сохранена в `city_currency_type_statistics.csv`.

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/04_initial_results.png]

Рисунок 4 — Первоначальные результаты пакетной обработки

8. Потоковая обработка

Чтение из Kafka. `spark.readStream.format("kafka")` с параметрами SSL:

- `subscribe=bank-transactions-v2`;
- `startingOffsets=earliest`;
- `failOnDataLoss=false` — retention тарифа Free удаляет старые сообщения;
- `maxOffsetsPerTrigger=5000`;
- `includeHeaders=true`.

Поле `value` (байты) приводится к строке и разбирается функцией `from_json` по **явной схеме** (все поля строковые плюс `_corrupt_record`). Битый JSON, массив или пустое сообщение попадают в `_corrupt_record` или дают пустую запись и отклоняются с причиной `invalid_json`. Дальше применяются **те же** функции нормализации и проверки, что и в пакетном этапе.

Запросы. Каждый запрос имеет собственный checkpoint и триггер 5 с.

- `accepted_stream` — `foreachBatch`:
  1. проверка;
  2. дедупликация внутри пакета и относительно уже обработанных транзакций (история и прошлые пакеты);
  3. запись принятых строк в `stream_transactions/batch_id=N` (перезапись только своего каталога — идемпотентно);
  4. запись отклонённых строк с причиной в `stream_rejected/`;
  5. пересчёт итоговых показателей «история + поток» и запись `updated_stream_statistics.csv` и журнала `stream_batches.csv`.
- `stream_type_counts` — потоковая агрегация с состоянием: `dropDuplicates(transaction_id)` → `groupBy(transaction_type)` → количество и сумма в EUR, `outputMode("complete")`, memory sink. Режим `complete` допустим, так как запрос содержит агрегацию.

Kafka **хранит** сообщения, а Spark обрабатывает их **микропакетами**: по триггеру он фиксирует в checkpoint диапазон новых offset’ов и обрабатывает его как небольшой DataFrame. Если новых сообщений нет, пакет с данными не создаётся. Перед отправкой статус потока был `Waiting for next trigger`, `isDataAvailable=False`, а нулевой пакет залогирован как пустой. `startingOffsets` действует только при пустом checkpoint. Удаление checkpoint вызывает повторное чтение с начала топика, но дубликаты по `transaction_id` не исказят итоги.

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/05_stream_started.png]

Рисунок 5 — Запуск потоковой обработки

Отправка. Пакет `B001` содержит 310 сообщений:

- 300 корректных транзакций;
- 5 некорректных: отрицательная сумма, нет `client_id`, неизвестная валюта, несуществующая дата, сумма-строка `"abc"`;
- 3 повтора;
- 2 строки битого JSON.

Ключ сообщения — `transaction_id`. Заголовки: `source`, `schema_version`, `content_type`, `send_batch`, `sequence`. Продюсер (`acks=all`) получил подтверждения на все 310 сообщений: партиция 0 — 156 сообщений (offset 0–155), партиция 1 — 154 (offset 0–153). [ЗАПОЛНИТЬ: распределение по партициям и offset'ы из вашего `kafka_delivery_report.csv`.]

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/06_messages_sent.png]

Рисунок 6 — Отправка новых транзакций (отчёт о доставке)

Таблица 9 — Журнал микропакетов (`stream_batches.csv`)

Микропакет	Получено	Принято	Отклонено	Причины
1 (новые сообщения)	310	300	10	invalid_json 2, missing_client_id 1, invalid_amount_format 1, non_positive_amount 1, unknown_currency 1, invalid_timestamp 1, duplicate_in_batch 3
2 (повторная отправка того же пакета)	310	0	310	duplicate_already_processed 300 и те же 10

Потоковая агрегация `stream_type_counts` после обработки: payment 133, transfer 84, withdrawal 42, deposit 29, refund 12 — всего 300 уникальных транзакций.

9. Сравнение первоначальных и обновлённых результатов

Показатели до отправки сохранены в `initial_stream_statistics.csv`, после обработки — в `updated_stream_statistics.csv`. Таблица изменений — `analysis_comparison.csv`.

Таблица 10 — Изменение показателей после поступления новых сообщений

Измерение	Значение	Было	Стало	Изменение	Сумма EUR: было → стало
total	all	490 174	490 474	+300	55 841 321,34 → 55 877 546,03
transaction_type	payment	205 050	205 183	+133	5 930 980,21 → 5 934 408,33
transaction_type	transfer	117 809	117 893	+84	20 296 693,78 → 20 310 267,28
transaction_type	withdrawal	78 037	78 079	+42	6 307 736,17 → 6 310 561,17

Измерение	Значение	Было	Стало	Изменение	Сумма EUR: было → стало
transaction_type	deposit	58 386	58 415	+29	22 503 593,41 → 22 519 684,39
transaction_type	refund	29 183	29 195	+12	609 704,08 → 610 011,17
currency	RUB / USD / EUR / CNY	—	—	+193 / +45 / +43 / +19	—

Число транзакций выросло ровно на число принятых (300). Повторная отправка того же пакета показатели не изменила: проверка «Показатели после повторной отправки не изменились: True». Итоги рассчитываются по уникальным `transaction_id` и не задваиваются. Демонстрация считается успешной, поскольку подтверждены доставка (отчёт продюсера и чтение из топика), обработка Spark (журнал микропакетов) и изменение показателей.

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/09_updated_results.png]

Рисунок 7 — Обновлённые результаты и таблица изменений

Результаты сохранены в Google Drive:

- `lab3_data/` — `raw_data.csv`, `new_transactions.jsonl/.csv`;
- `lab3_results/` — 20 CSV, `execution_log.txt`, `environment.json`;
- `lab3_intermediate/` — `clean_history.parquet`, `work_backup` с checkpoint и хранилищем потока.

Повторный запуск всех ячеек проверен: генерация пропускается, checkpoint восстанавливается из копии, номера микропакетов продолжаются, а самопроверка блокнота проходит (16 из 16).

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/10_drive_results.png]

Рисунок 8 — Сохранённые результаты в Google Drive

10. Проверка доставки сообщений в Aiven

Доставка подтверждена тремя способами:

1. **Отчёт продюсера** (`kafka_delivery_report.csv`): для каждого сообщения — ключ, партиция, offset, timestamp брокера, отсутствие ошибки.
2. **Независимое чтение из топика** (`kafka_messages_sample.csv`): отдельный консьюмер без фиксации offset’ов читает сообщения с

отправленных offset’ов и выводит partition, offset, timestamp, headers, key и value. Прочитано 310 из 310.

3. **Aiven UI**: сервис → Topics → `bank-transactions-v2` → Messages → Fetch messages (формат JSON).
   - Видны partition, offset, timestamp, key, headers и payload.
   - Просмотр требует включённого Apache Kafka REST API (Karapace).
   - Общее число сообщений отображается в списке топиков и в Metrics.

[ЗАПОЛНИТЬ: укажите, был ли доступен просмотр сообщений на вашем тарифе.]

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/07_aiven_messages.png]

Рисунок 9 — Список сообщений топика в Aiven UI

[МЕСТО ДЛЯ СКРИНШОТА: screenshots/08_aiven_payload_metadata.png]

Рисунок 10 — Payload и метаданные сообщения в Aiven UI

Проверки. Автотесты проекта (40) подтверждают:

- корректность генерации, обнаружения пропусков и дубликатов, очистки и агрегатов;
- работу SSL-хранилищ;
- обработку граничных случаев: пустой пакет, некорректный JSON, отсутствующее поле, повторное сообщение, повтор микропакета, недоступный брокер.

Сквозной тест Kafka по SSL и Structured Streaming выполнен на локальном брокере с обязательной клиентской аутентификацией. Проверки с реальными Aiven и Google Drive выполняются самопроверкой блокнота в Colab (16 пунктов, `notebook_checks.csv`). [ЗАПОЛНИТЬ: результат самопроверки в Colab.]

11. Выводы

1. Сгенерирован воспроизводимый набор из 505 000 банковских транзакций с пропусками (≈ 2 %), дубликатами (≈ 1 %) и некорректными значениями (≈ 1 %). Явная схема со строковыми полями и безопасное приведение типов позволили обнаружить все проблемы без аварийного завершения. После очистки по прозрачным правилам осталось 490 174 строки, баланс удалённых и оставшихся строк сходится.
2. Пакетный анализ показал преобладание платежей (42 %) и операций в рублях (62 %). Наибольший средний чек — у пополнений и переводов, крупнейший город по числу и сумме операций — Москва. Распределение сумм сильно скошено вправо.
3. Настроено защищённое подключение к Kafka в Aiven:
   - Python-клиент использует PEM-файлы;
   - Spark (JVM) — хранилища PKCS12, созданные OpenSSL и keytool после проверки соответствия ключа и сертификата;
   - секреты не попадают в Google Drive, код и вывод.
4. Spark Structured Streaming обрабатывает сообщения Kafka микропакетами с интервалом 5 с:
   - некорректные сообщения (битый JSON, нет обязательного поля, неверные значения, повторы) отклоняются с указанием причины;
   - принятые транзакции сохраняются в устойчивом хранилище;
   - итоговые показатели «история + поток» пересчитываются автоматически после каждого микропакета.
5. После отправки 300 корректных транзакций общее число транзакций выросло с 490 174 до 490 474. Повторная отправка не изменила показатели, что подтверждает идемпотентность обработки. Checkpoint сохраняется в Google Drive и позволяет продолжить поток в новой

сессии Colab. Строгая сквозная гарантия exactly-once для всего конвейера не заявляется.

6. Ограничения: Colab — одна ВМ (`local[*]`), бесплатный Aiven ограничен двумя партициями на топик и выключается при простое, курсы валют условные.

12. Ответы на контрольные вопросы

Важно. Методичка не была приложена к заданию, поэтому точные формулировки 15 контрольных вопросов неизвестны. Ниже приведены 15 вопросов, покрывающих тему работы: Apache Kafka, Aiven, SSL, Spark Structured Streaming, объединение потока с историей. **Сверьте их с методичкой.** Если формулировки отличаются, ответы, скорее всего, найдутся среди приведённых: темы совпадают с обязательными разделами работы.

1. Что такое Apache Kafka и из каких основных компонентов состоит?

Apache Kafka — распределённая платформа потоковой передачи событий: долговечный, реплицируемый журнал сообщений с публикацией и подпиской. Компоненты:

- **брокеры** — серверы, хранящие данные;
- **топики** — именованные потоки сообщений;
- **партиции** — упорядоченные журналы, на которые делится топик;
- **продюсеры** — пишут сообщения;
- **консьюмеры** и **группы консьюмеров** — читают сообщения;
- **контроллер** кластера — в новых версиях KRaft вместо ZooKeeper.

Сообщение (запись) состоит из ключа, значения (payload), заголовков (headers) и метки времени. В работе продюсер `producer.py` публикует транзакции в топик `bank-transactions-v2` сервиса Aiven, а Spark Structured Streaming выступает потребителем.

2. Что такое топик и партиция? Как Kafka обеспечивает масштабирование и порядок сообщений?

Топик — логическая категория сообщений. Физически он разбит на **партиции** — упорядоченные журналы, где каждое сообщение получает возрастающий **offset**.

- Партиции распределяются по брокерам и читаются параллельно: каждой партиции в группе соответствует не более одного консьюмера. Так Kafka масштабируется.
- Порядок гарантируется **только внутри партиции**. Сообщения с одинаковым ключом попадают в одну партицию (хеш ключа), поэтому порядок по ключу сохраняется.

В работе топик имеет 2 партиции (максимум на тарифе Free), ключ сообщения — `transaction_id`. В отчёте о доставке видно, как 310 сообщений распределились по партициям 0 и 1.

3. Что такое offset и как Kafka хранит сообщения? Чем Kafka отличается от классической очереди?

Offset — порядковый номер сообщения в партиции. Kafka **хранит** сообщения на диске в течение заданного срока или объёма (**retention**) независимо от того, прочитаны ли они.

- Консьюмер сам определяет, с какого offset читать. Поэтому одни и те же данные могут независимо читать несколько потребителей, а при необходимости — перечитать.
- В классической очереди сообщение обычно удаляется после подтверждения потребителем.

В работе это используется так:

- Spark и проверочный консьюмер (`read_messages`) читают одни и те же сообщения независимо;
- `startingOffsets="earliest"` позволяет обработать сообщения, отправленные до запуска потока.

4. Что такое фактор репликации, лидер партиции и ISR? Как Kafka обеспечивает отказоустойчивость?

- **Фактор репликации** — число копий партиции на разных брокерах.
- **Лидер** обслуживает запись и чтение, **последователи** копируют его журнал.
- **ISR** (in-sync replicas) — реплики, не отставшие от лидера. При отказе лидера новый выбирается из ISR.
- При `acks=all` продюсер получает подтверждение только после записи во все реплики ISR, а `min.insync.replicas` задаёт их минимальное число.

На бесплатном тарифе Aiven конфигурация кластера фиксирована. Фактический фактор репликации топика виден в Aiven Console и в выводе `check_connection` (`replicas_per_partition`). Продюсер в работе использует `acks=all`.

5. Как работает продюсер и какие гарантии доставки возможны?

Продюсер сериализует ключ и значение, выбирает партицию (по хешу ключа), группирует сообщения в пакеты (`linger.ms`, сжатие) и отправляет их лидеру партиции.

Семантики доставки:

- **at-most-once** — без повторов, возможна потеря;
- **at-least-once** — повторы при сбоях, возможны дубликаты;
- **exactly-once** — идемпотентный продюсер и транзакции Kafka, плюс согласованный потребитель.

В работе:

- `acks=all`;
- обработчик `on_delivery` для каждого сообщения возвращает partition, offset и timestamp брокера или ошибку;
- `flush()` с таймаутом: неподтверждённые сообщения дают понятную ошибку.

Возможные дубликаты при повторной отправке устраняются на стороне Spark по `transaction_id`.

6. Что такое группа консьюмеров и как хранятся offset’ы? Как это делает Spark?

Группа консьюмеров делит партиции топика между участниками и фиксирует прочитанные offset’ы во внутреннем топике `__consumer_offsets`.

Spark Structured Streaming не полагается на фиксацию offset’ов в группе. Для каждого микропакета он сам определяет диапазон offset’ов и записывает его в **checkpoint** (`offsets/`, `commits/`). После перезапуска чтение продолжается с позиции из checkpoint.

В работе у каждого потока свой checkpoint (`checkpoints/accepted_stream`, `checkpoints/stream_type_counts`). Проверочный консьюмер использует временную группу без автокоммита и не влияет на Spark.

7. Что такое Aiven и в чём преимущества и ограничения управляемого Kafka?

Aiven — облачная платформа управляемых (managed) сервисов с открытым исходным кодом. Провайдер берёт на себя:

- развёртывание брокеров;
- TLS-сертификаты;
- обновления;
- мониторинг;
- резервирование.

Пользователь получает адрес подключения и сертификаты и создаёт топики в веб-интерфейсе.

Ограничения бесплатного тарифа:

- до 5 топиков по 2 партиции;
- пропускная способность около 250 КиБ/с;
- фиксированные настройки (в том числе retention);
- нет Kafka Connect и SLA;

- автоматическое выключение при простое (около 24 ч).

Для учебной задачи (сотни сообщений) этих ресурсов достаточно, но перед работой нужно убедиться, что сервис в состоянии Running.

8. Как организовано защищённое подключение к Kafka в Aiven? Зачем keystore и truststore?

Aiven шифрует трафик TLS и аутентифицирует клиентов:

- **сертификатом** (mTLS): `service.cert` и `service.key`, подписанные CA проекта (`ca.pem`);
- или **SASL/SCRAM** (логин и пароль) поверх TLS.

Каждый клиент использует сертификаты по-своему:

- Python-клиент (librdkafka) принимает PEM-файлы напрямую.
- Spark Kafka connector работает в **JVM**, и `kafka-clients` загружает ключи из хранилищ Java:
  - **truststore** содержит доверенный CA (проверка сертификата брокера);
  - **keystore** — клиентский сертификат и закрытый ключ (предъявляется брокеру).

В работе хранилища создаются в формате **PKCS12**: `keytool -importcert` и `openssl pkcs12 -export`. Пароли случайные на сессию и передаются через переменные окружения. Перед этим проверяется, что ключ соответствует сертификату и сертификат подписан CA.

9. Что такое Spark Structured Streaming и модель микропакетов?

Structured Streaming рассматривает поток как **неограниченную таблицу**, к которой постоянно добавляются строки. Запрос пишется тем же DataFrame API, что и пакетный, а Spark выполняет его **инкрементально**.

В режиме микропакетов по **триггеру** (в работе — каждые 5 с):

1. Spark определяет новые offset’ы в Kafka;
2. записывает их в checkpoint;
3. обрабатывает полученный небольшой DataFrame и фиксирует результат.

Если новых сообщений нет, микропакет с данными не создаётся (статус `isDataAvailable=False`). Kafka при этом продолжает хранить сообщения, а Spark лишь читает их порциями.

10. Какие режимы вывода (output modes) есть и как они сочетаются с агрегациями?

- **append** — выводятся только новые строки. Для агрегаций допускается только с водяным знаком (watermark), иначе строки результата могли бы ещё измениться.
- **update** — выводятся только изменившиеся строки результата.
- **complete** — каждый раз выводится весь результат. Допустим **только для запросов с агрегацией**, так как весь результат хранится в состоянии.

В работе:

- поток `stream_type_counts` (`groupBy` по типу операции) использует `complete` с memory sink;
- поток `accepted_stream` (без агрегации, `foreachBatch`) использует режим по умолчанию `append`.

11. Для чего нужен checkpoint? Как связаны `startingOffsets`, checkpoint и `failOnDataLoss`?

Checkpoint хранит:

- обработанные offset’ы;
- журнал завершённых микропакетов;
- состояние агрегаций.

Это позволяет после сбоя или перезапуска продолжить с того же места.

`startingOffsets` применяется **только** при первом запуске с пустым checkpoint:

- `earliest` — с начала топика;
- `latest` — только новые сообщения, при этом отправленные до запуска будут пропущены.

При существующем checkpoint параметр игнорируется. Удаление checkpoint приводит к повторной обработке с `startingOffsets`. Изменение запроса может сделать старый checkpoint несовместимым.

`failOnDataLoss=false` нужен, потому что сообщения могли быть удалены по retention: offset'ы в checkpoint становятся недоступны, и вместо ошибки чтение продолжается.

Как это устроено в работе:

- checkpoint не удаляется при повторном запуске;
- после остановки потоков он копируется в Google Drive;
- в новой сессии Colab восстанавливается — проверено: номера микропакетов продолжились.

12. Чем время события отличается от времени обработки? Что такое watermark и окна?

- **Время события** (event time) — момент операции (поле `timestamp` транзакции).
- **Время обработки** (processing time) — момент обработки в Spark.
- Из-за задержек данные приходят с опозданием и не по порядку.
- **Окна** (`window(timestamp, "10 minutes")`) группируют события по времени события.
- **Watermark** (`withWatermark("timestamp", "1 hour")`) задаёт допустимое опоздание: более старое состояние можно удалить, а окна — окончательно вывести в режиме `append`.

В работе агрегаты строятся по категориям, а не по окнам, поэтому watermark не нужен. Для обработки по времени операции его следует добавить, чтобы состояние не росло бесконечно.

13. Как корректно объединить потоковые данные с историческими?

Возможны три подхода:

- **stream-static join** — соединение потока со статическим DataFrame, например со справочником;

- агрегация с состоянием в самом потоке;
- запись потока в **устойчивое хранилище** и пересчёт итогов по «истории + накопленному потоку».

Простой `union` статического и потокового DataFrame внутри одного запроса **не сохраняет** объединённую историю между перезапусками и не даёт итоговых показателей по всем данным.

В работе выбран третий подход:

1. очищенная история хранится в Parquet;
2. в `foreachBatch` принятые транзакции пишутся в `stream_transactions/batch_id=N`;
3. итоговые показатели пересчитываются как агрегаты истории плюс агрегаты всего сохранённого потока;
4. повтор микропакета перезаписывает только свой каталог, повторы транзакций отбрасываются по `transaction_id`.

Поэтому итоги не задваиваются: повторная отправка того же пакета их не изменила.

14. Чем пакетная обработка отличается от потоковой? Когда что использовать?

Пакетная обработка работает с **ограниченным** набором данных целиком. Её достоинства — простота, полный контекст, сложные объединения и статистика (медиана, топы). Пример — анализ 505 000 исторических строк.

Потоковая обработка непрерывно обрабатывает **неограниченный** поток с малой задержкой, инкрементально и с состоянием. Её сложности — опоздания, дубли, контрольные точки. Пример — новые транзакции из Kafka.

Распространённая схема сочетает оба подхода: пакетный расчёт по истории плюс инкрементальное обновление по потоку. Это реализовано в работе. В архитектурной терминологии ей близки Lambda (отдельные

пакетный и потоковый слои) и Kappa (всё как поток с возможностью переигрывания журнала Kafka).

15. Как обеспечить качество данных и надёжность потоковой обработки? Какие гарантии даёт конвейер?

Меры качества данных, применённые в работе:

- явная схема JSON;
- безопасное приведение типов (`try_cast`, `try_to_timestamp`);
- проверка обязательных полей и допустимых значений;
- отдельное хранилище отклонённых сообщений с причиной (аналог dead-letter queue);
- дедупликация внутри микропакета и относительно уже обработанных данных.

В Spark источник Kafka в сочетании с checkpoint позволяет переигрывать микропакеты. Поэтому сквозная семантика «ровно один раз» достигается, только если приёмник **идемпотентен** или транзакционен.

В работе идемпотентны:

- запись хранилища по `batch_id`;
- итоговые показатели, которые пересчитываются по уникальным `transaction_id`.

Вспомогательные файлы — журнал CSV и копия в Drive — обновляются по принципу «как минимум один раз». Поэтому строгая гарантия exactly-once для **всего** конвейера не заявляется. Корректность итогов подтверждена тестами: повтор микропакета и повторная отправка сообщений не меняют показатели.

Список использованных источников

Дата обращения для всех ресурсов — 19.10.2026.

1. Apache Spark. Structured Streaming Programming Guide (Spark 4.0.4) [Электронный ресурс]. — URL: https://spark.apache.org/docs/4.0.4/streaming/index.html

2. Apache Spark. Structured Streaming + Kafka Integration Guide [Электронный ресурс]. — URL: https://spark.apache.org/docs/4.0.4/streaming/structured-streaming-kafka-integration.html
3. Apache Spark. ANSI Compliance [Электронный ресурс]. — URL: https://spark.apache.org/docs/4.0.4/sql-ref-ansi-compliance.html
4. PySpark 4.0.4. Installation [Электронный ресурс]. — URL: https://spark.apache.org/docs/4.0.4/api/python/getting_started/install.html
5. Apache Kafka. Documentation [Электронный ресурс]. — URL: https://kafka.apache.org/documentation/
6. Aiven. Aiven for Apache Kafka® free tier [Электронный ресурс]. — URL: https://aiven.io/docs/products/kafka/free-tier/kafka-free-tier
7. Aiven. Create a free tier Aiven for Apache Kafka® service [Электронный ресурс]. — URL: https://aiven.io/docs/products/kafka/free-tier/create-free-tier-kafka-service
8. Aiven. Create Apache Kafka® topics [Электронный ресурс]. — URL: https://aiven.io/docs/products/kafka/howto/create-topic
9. Aiven. Connect to Aiven for Apache Kafka® with Python [Электронный ресурс]. — URL: https://aiven.io/docs/products/kafka/howto/connect-with-python
10. Aiven. Configure Java SSL keystore and truststore to access Apache Kafka® [Электронный ресурс]. — URL: https://aiven.io/docs/products/kafka/howto/keystore-truststore
11. Aiven. Enable schema registry and REST proxy (Karapace) [Электронный ресурс]. — URL: https://aiven.io/docs/products/kafka/karapace/howto/enable-karapace
12. Confluent. confluent-kafka-python documentation [Электронный ресурс]. — URL: https://docs.confluent.io/platform/current/clients/confluent-kafka-python/html/index.html

13. Google Colab. Runtime versions (FAQ) [Электронный ресурс]. — URL: https://research.google.com/colaboratory/runtime-version-faq.html
14. googlecolab/backend-info — состав образа Colab [Электронный ресурс]. — URL: https://github.com/googlecolab/backend-info
15. ArchWiki. Docker [Электронный ресурс]. — URL: https://wiki.archlinux.org/title/Docker